

Fase 1 — Visão vs Código

Comparação entre o que a Fluxa deveria ser (documentação fundacional) e o que ela é hoje (código em produção).

Classificação em 4 estados:

- ✔ Implementado e alinhado — Funciona e corresponde à visão
- ⚠ Implementado, mas divergente — Existe no código, mas difere da visão
- ◆ Parcialmente implementado — Começou, mas incompleto
- ✖ Ainda não implementado — Existe na visão, não existe no código

Resumo Executivo

Estado	Quantidade	%
✔ Implementado e alinhado	28	52%
⚠ Implementado, mas divergente	4	7%
◆ Parcialmente implementado	12	22%
✖ Ainda não implementado	10	19%
Total	54	100%

Conclusão: O produto está 52% alinhado com a visão ideal. Os 22% parciais são os que mais precisam de atenção — são features que existem mas não entregam o valor completo prometido.

1. INFRAESTRUTURA OPERACIONAL

Pedidos e Pagamentos

Funcionalidade	Estado	Detalhe
Pedido via QR Code (celular)	✅ Implementado e alinhado	Menu público, carrinho, checkout completo
Pagamento via Pix (MP)	✅ Implementado e alinhado	Produção, QR gerado, webhook confirma
Pagamento via Cartão (MP)	✅ Implementado e alinhado	Cartão real processado pelo MP
Pagamento via Cartão (Stripe)	✅ Implementado e alinhado	Sandbox funcional, pronto para live
Split automático (dinheiro direto pro produtor)	✅ Implementado e alinhado	MP Marketplace OAuth + Stripe Connect
Verificação de saldo em tempo real	⚠️ Implementado, mas divergente	Não existe “saldo” — é pagamento por pedido (correto pela filosofia “sem carteira”)
Modo offline (staff)	✅ Implementado e alinhado	useOfflineStaff hook + sync + localStorage queue
Pedidos offline (fiado)	✅ Implementado e alinhado	offlineOrders router completo com pagamento posterior

KDS (Kitchen Display System)

Funcionalidade	Estado	Detalhe
Tela do bar (recebe pedidos)	✅ Implementado e alinhado	StaffKDS.tsx + StationKDS.tsx
Atualização de status (preparando/pronto)	✅ Implementado e alinhado	updateStatus mutation
Push notification para staff	✅ Implementado e alinhado	webpush + staffPushSubscriptions
Push notification para cliente	✅ Implementado e alinhado	Notifica quando pedido fica pronto
Múltiplos pontos de venda	✅ Implementado e alinhado	salePoints + stationTokens

Estoque

Funcionalidade	Estado	Detalhe
Controle de estoque por produto	✅ Implementado e alinhado	stockMovements + AdminStockMonitor
Alerta de ruptura	✅ Implementado e alinhado	lowAlerts + notifyLowStock
Ficha técnica (ingredientes)	✅ Implementado e alinhado	ingredients router + productIngredients
Reconciliação pós-evento	✅ Implementado e alinhado	AdminStockReconciliation.tsx
Previsão de estoque (IA)	🔴 Parcialmente implementado	generateStockForecast existe mas usa regras simples, não ML real

2. SYNTROPY (CÉREBRO)

Planejamento (Antes do Evento)

Funcionalidade	Estado	Detalhe
Conversa natural com produtor	✔ Implementado e alinhado	syntropyPlanner.chat — LLM conversacional
Geração de plano operacional	✔ Implementado e alinhado	Gera público, faturamento, equipe, estoque, cardápio, riscos
Criação de evento a partir do plano	✔ Implementado e alinhado	“Criar evento com este plano” funcional
Simulação de cenários	◆ Parcialmente implementado	LLM responde “e se chover?” mas não tem modelo quantitativo
Previsão climática	◆ Parcialmente implementado	Open-Meteo integrado no pós-evento, mas não no planejamento
Perfil do público (pré-evento)	✗ Ainda não implementado	Não analisa compradores de ingressos antes do evento

Operação (Durante o Evento)

Funcionalidade	Estado	Detalhe
Smart Routing (distribuição de pedidos)	✔ Implementado e alinhado	dispatch.ts completo com score por fila + tempo
Mission Control (dashboard ao vivo)	✔ Implementado e alinhado	MissionControl.tsx + AdminFluxaControl
Alertas inteligentes	✔ Implementado e alinhado	intelligence-engine.ts detecta anomalias + gera sugestões
Identificação de gargalos	✔ Implementado e alinhado	detectAnomaliesRules + detectAnomaliesLLM
Promoções automáticas	◆ Parcialmente implementado	promos router existe, mas Syntropy não sugere automaticamente
Recomendação abrir/fechar bar	◆ Parcialmente implementado	Alertas existem, mas não há ação automática
Acompanhamento receita vs projeção	◆ Parcialmente implementado	Receita ao vivo existe, projeção pré-evento não é comparada em tempo real

Análise (Depois do Evento)

Funcionalidade	Estado	Detalhe
Fluxa Vision (relatório executivo)	✔ Implementado e alinhado	postEventReport com LLM gerando análise narrativa
Faturamento vs projetado	✔ Implementado e alinhado	previousEventComparison no relatório
Produtos mais lucrativos	✔ Implementado e alinhado	topSold + margem calculada
Comportamento do público	◆ Parcialmente implementado	Horário de pico sim, perfil demográfico não
Comparação histórica	◆ Parcialmente implementado	Compara com evento anterior, mas não com benchmark coletivo
Recomendações para próximo evento	✔ Implementado e alinhado	LLM gera recomendações com impacto estimado

Aprendizado Contínuo

Funcionalidade	Estado	Detalhe
Baselines por evento	✔ Implementado e alinhado	calculateBaselines + iaBaselines table
Aprendizado individual (por produtor)	◆ Parcialmente implementado	Dados existem, mas não alimentam o próximo planejamento automaticamente
Aprendizado coletivo (plataforma)	✗ Ainda não implementado	Cada produtor é isolado, sem inteligência cruzada
Benchmark entre eventos similares	✗ Ainda não implementado	benchmark endpoint retorna dados do próprio produtor

3. MÓDULOS DE NEGÓCIO

Ingressos

Funcionalidade	Estado	Detalhe
Criação de tipos de ingresso	✅ Implementado e alinhado	ticketTypes + tickets router
Venda de ingressos	✅ Implementado e alinhado	TicketPurchase.tsx com pagamento
Check-in no evento	⚠️ Implementado, mas divergente	eventEntries table existe mas fluxo de check-in via QR não está claro no frontend
Lotes (early bird, regular, VIP)	💎 Parcialmente implementado	Tipos existem, mas sem lógica de esgotamento automático entre lotes

Financeiro

Funcionalidade	Estado	Detalhe
Dashboard financeiro	✅ Implementado e alinhado	AdminFinancials.tsx + AdminRecebimentos
Split automático MP	✅ Implementado e alinhado	createPixPaymentMarketplace
Split automático Stripe	✅ Implementado e alinhado	Stripe Connect onboarding
Faturas/cobranças da plataforma	✅ Implementado e alinhado	AdminFaturas.tsx + platformRevenue
Saques (withdrawal)	✅ Implementado e alinhado	withdrawal router + StaffWithdrawal

CRM

Funcionalidade	Estado	Detalhe
Perfis de participantes	◆ Parcialmente implementado	participantProfiles table + syncProfiles, mas sem interface rica
Histórico de consumo por pessoa	◆ Parcialmente implementado	Dados existem nos pedidos, mas não consolidados em perfil
Programa de fidelidade	✗ Ainda não implementado	Não existe sistema de pontos ou recompensas
Comunicação com participantes	✗ Ainda não implementado	Sem email marketing ou push pós-evento

Contratos

Funcionalidade	Estado	Detalhe
Geração de contrato digital	✓ Implementado e alinhado	contracts router com geração automática por plano
Assinatura digital	✓ Implementado e alinhado	ContractSign.tsx com hash + timestamp

Segurança

Funcionalidade	Estado	Detalhe
MFA (autenticação 2 fatores)	✓ Implementado e alinhado	mfa router com TOTP
Audit logs	✓ Implementado e alinhado	auditLogs com ação, severidade, IP
Rate limiting	✓ Implementado e alinhado	60 req/min em produção
HSTS + CSP	✓ Implementado e alinhado	Headers de segurança aplicados

4. INTERFACES

Interface	Estado	Detalhe
Landing page (fluxa.bar)	✓ Implementado e alinhado	Proposta clara, planos, demo, CTA
Onboarding com Syntropy	✓ Implementado e alinhado	Conversa natural + geração de plano
Dashboard admin	✓ Implementado e alinhado	DashboardLayout com sidebar completa
Mission Control (ao vivo)	✓ Implementado e alinhado	Receita, filas, alertas, top produtos
Menu público (participante)	✓ Implementado e alinhado	Cardápio + carrinho + checkout
KDS (operador)	✓ Implementado e alinhado	StaffKDS + StationKDS
Demo interativa	✓ Implementado e alinhado	DemoShowcase + LiveDemo
App nativo (mobile)	✗ Ainda não implementado	Apenas PWA responsiva

5. VISÃO DE FUTURO (ainda não implementado)

Funcionalidade	Prioridade	Complexidade
Aprendizado coletivo (inteligência cruzada)	Alta	Alta
Precificação dinâmica	Média	Média
Marketplace de fornecedores	Média	Alta
IA conversacional para todos os atores	Média	Média
Programa de fidelidade	Média	Baixa
Comunicação pós-evento (email/push)	Alta	Baixa
App nativo	Baixa	Alta
Benchmark nacional	Baixa (precisa volume)	Média
Operação parcialmente autônoma	Baixa (2030)	Muito Alta
API pública	Baixa	Média

6. DIVERGÊNCIAS CRÍTICAS (⚠)

1. “Saldo” vs “Pagamento por pedido”

Visão original (project_instructions): Fala em “saldo”, “recarga”, “verificar saldo”.

Código atual: Não existe carteira/saldo. Cada pedido é pago individualmente (Pix ou cartão).

Veredicto: O código está **correto** segundo a filosofia atual (“nunca teremos carteira digital”). A documentação de projeto precisa ser atualizada para remover referências a saldo/recarga.

2. Check-in de ingressos

Visão: Participante entra no evento com ingresso digital.

Código: Table `eventEntries` existe, mas o fluxo de check-in via QR no frontend não está evidente. Pode existir via `EventEntry.tsx` mas precisa validação.

3. Promoções “automáticas”

Visão: Syntropy sugere e executa promoções automaticamente.

Código: Promoções existem (CRUD completo), mas são criadas manualmente pelo admin. A Syntropy não sugere promoções proativamente durante o evento.

4. Previsão climática no planejamento

Visão: Syntropy ajusta previsões baseado no clima esperado.

Código: Open-Meteo está integrado no pós-evento (análise histórica), mas não no planejamento (previsão futura). O syntropyPlanner não consulta previsão do tempo.

7. PRÓXIMOS PASSOS RECOMENDADOS

Prioridade 1 — Alinhar (corrigir divergências)

1. Atualizar project_instructions removendo referências a “saldo/recarga”
2. Validar fluxo de check-in de ingressos no frontend
3. Conectar previsão climática ao syntropyPlanner (Open-Meteo forecast API)

Prioridade 2 — Completar parciais (maior impacto)

1. Promoções sugeridas pela Syntropy (usar intelligence-engine para detectar oportunidade → gerar sugestão)
2. Aprendizado individual: alimentar syntropyPlanner com dados de eventos anteriores do mesmo produtor
3. Comparação receita projetada vs real em tempo real no Mission Control

Prioridade 3 — Implementar novos (visão de futuro)

1. Comunicação pós-evento (push/email com Vision resumido)
 2. Programa de fidelidade básico (frequência → benefício)
 3. Aprendizado coletivo (modelo agregado cross-produtor)
-

8. MAPA DE ARQUIVOS POR FUNCIONALIDADE

Funcionalidade	Arquivos principais
Pedidos	<code>server/routers/orders.ts</code> , <code>client/src/pages/Menu.tsx</code>
Pagamento MP	<code>server/mercadopago.ts</code> , <code>server/routers/orders.ts</code>
Pagamento Stripe	<code>server/stripe.ts</code> , <code>client/src/components/StripePaymentForm.tsx</code>
Split	<code>server/routers/stripeConnect.ts</code> , <code>server/mercadopago.ts</code>
Smart Routing	<code>server/dispatch.ts</code>
KDS	<code>client/src/pages/StaffKDS.tsx</code> , <code>client/src/pages/StationKDS.tsx</code>
Estoque	<code>server/routers.ts</code> (stock) , <code>client/src/pages/AdminStockMonitor.tsx</code>
Ingredientes	<code>server/routers/ingredients.ts</code>
Intelligence	<code>server/intelligence-engine.ts</code> , <code>server/routers/intelligence.ts</code>
Syntropy Planner	<code>server/routers/syntropyPlanner.ts</code>
Vision (pós-evento)	<code>server/routers.ts</code> (postEventReport) , <code>client/src/pages/AdminPostEvent.tsx</code>
Mission Control	<code>server/routers/fluxaControl.ts</code> , <code>client/src/pages/MissionControl.tsx</code>
Ingressos	<code>server/routers/tickets.ts</code> , <code>client/src/pages/TicketPurchase.tsx</code>
Promoções	<code>server/routers.ts</code> (promos) , <code>client/src/pages/AdminPromos.tsx</code>
Contratos	<code>server/routers.ts</code> (contracts) , <code>client/src/pages/ContractSign.tsx</code>

Funcionalidade	Arquivos principais
CRM	<code>server/routers.ts (crm)</code> , <code>drizzle/schema.ts (participantProfiles)</code>
Offline	<code>client/src/hooks/useOfflineStaff.ts</code> , <code>server/routers/offlineOrders.ts</code>
Push	<code>server/pushNotification.ts</code>
Leads	<code>server/routers.ts (leads)</code>
Regional	<code>server/routers.ts (regionalAnalytics)</code>
MFA	<code>server/routers.ts (mfa)</code>
Open Bar	<code>server/routers.ts (openBar)</code>